LifeDocs Press

GODSHAND Starter Manual

A practical entry guide to cinematic AI filmmaking

From single portrait to finished AI film

A LifeDocs Press Publication

Build Map and Source of Truth

The canonical map of the kit.

🧭

GODSHAND Kit — Build Map & Source of Truth

🟠 **Correction to my earlier note on this page — you were right.** My first pass said "every file is byte-identical, nothing was lost." That was wrong, and I am fixing the record. The core rendering **engine** (all of `src/`, every workflow, `run.py`, the configs) *is* byte-for-byte identical in every build — that part is safe. **But the builds are NOT all equal:** two builds dropped whole features, and one build's launcher was silently **regressed** to an inferior version. Your instinct that "we lost more than the folder" was correct.

✅ What is safe (identical in every build)

The actual engine never changed. These are byte-for-byte identical across FINAL_, TUNED_AK, and FINAL_AK:

- All of `src/` (`render_motion.py`, `forge_character.py`, `voice_pass.py`, `score_gen.py`, `stitch.py`, `postprocess.sh`, …)
- All `workflows/*.json`, `run.py`, and every `config_*.yaml` / `shots_*.json`
- All character reference images in `refs/<name>/face` and `/body`

Nothing about how your film actually renders was corrupted. The damage is only in the **launchers, docs, and scaffolding** that wrap the engine.

⚠️ Feature audit — what was DROPPED or OVERRIDDEN (25 Jul 2026)

Compared file-by-file by content hash, with packaging nesting normalized.

Feature	FINAL_ (your Downloads)	TUNED_AK	FINAL_AK
:--	:-:	:-:	:-:
`NEW_STORY_TEMPLATE/` (author any new film)	✅	❌ dropped	❌ dropped
`GODSHAND_ONECLICK.sh` (main launcher)	✅ full (10,565 B)	❌ dropped	⚠️ **regressed** (9,120 B)
`README_L4.md`	✅ full (3,437 B)	❌ dropped	⚠️ **regressed** (3,129 B)
`l4_model_setup.py` (WAN 2.2 downloader)	✅	❌ dropped	✅
`notebooks/` (2 Kaggle notebooks)	✅	❌ dropped	✅
`RUN_TUNED.sh` (one-command tuned launcher)	❌	✅ only here	❌

🚨 The important one: the launcher was REGRESSED in FINAL_AK

`GODSHAND_ONECLICK.sh` exists in two versions. **The good one is in your Downloads (FINAL_).** The FINAL_AK copy is 1,445 bytes smaller and was rewritten in a way that reintroduces the exact "wrong folder" behavior you have been fighting:

Good version (FINAL_ / Downloads):

- Story is **selectable** — `KIT_CONFIG` picks any film (not just the confrontation).
- Characters are read **dynamically** from the config's `character_forge.targets`.
- Character photos are **OPTIONAL** — if `kit/refs/<name>/face` has images it uses them, otherwise it **auto-forges** a consistent face, logging the choice per character.
- `GODSHAND_ANCHORS` images are copied into `kit/refs/` (the correct home).
- Explicitly states: *"There is no `kit/anchors/` folder — the real location is `kit/refs/<name>/`."*

Regressed version (FINAL_AK):

- **Hardcoded** to the confrontation film + "26 shots" — loses the any-story engine.
- Drops the per-character auto-forge detection.
- Reintroduces a phantom `kit/anchors/` folder and dumps anchor images into `ComfyUI/input/` and the **kit root** — this is the stray-image / wrong-location behavior that caused the Spider-Man / Joker contamination.
- Its `README_L4.md` tells you to use the non-existent `kit/anchors/`.

➡️ **Use the launcher and README from FINAL_ (Downloads). Do not use the FINAL_AK versions.**

✅ Which build to use (corrected)

`GODSHAND_FINAL_` **in your Downloads is your MASTER build.** It has the correct launcher, `NEW_STORY_TEMPLATE/`, the notebooks, `l4_model_setup.py`, and the full README — everything, in its correct form.

The only thing it lacks is `RUN_TUNED.sh` (one-command tuned launcher), and you do not strictly need it — its tuned values are already baked into `NEW_STORY_TEMPLATE/config_newstory.yaml` (`denoise: 0.55`, `codeformer_fidelity: 0.5`, `active_profile: lightning_l4`). To also get the one-command launcher, copy it in:

```
cp GODSHAND_TUNED_AK/RUN_TUNED.sh GODSHAND_FINAL/kit/
```

> Correction: I earlier told you to use `TUNED_AK` as the "richest" build. That was wrong — TUNED_AK actually dropped the launcher, `l4_model_setup.py`, the notebooks, and `NEW_STORY_TEMPLATE/`. FINAL_ (Downloads) is the right master.

🕷️ Root cause — why Spider-Man / Joker appeared as “jack” (the original question)

Three things had to line up. All three are now confirmed in the code:

1. **Two stray images were sitting in jack’s staging folder.**
   `refs/jack/_inbox/jack_preface.jpg` is a Spider-Man (Tobey Maguire) still; `refs/jack/_inbox/joker.png` is a Joker still. `_inbox/` is a “dump a pile of candidate

photos here" staging area that `cull_photos.py` is meant to sort into `face/` and `body/` — but it was never run on jack, so the strays stayed.

2. **The reference loader reads recursively.** `comfy_client.py` and `render_motion.py` collect refs with `glob("refs/jack/**/*", recursive=True)` — so they reach **into** `_inbox/` instead of reading only `refs/jack/face/`.
3. `_inbox` **sorts first, and only the first 5 refs are used.** An underscore sorts before letters, so the recursive list starts with the two `_inbox` images. The code keeps `refs[:5]`, so **Spider-Man and Joker became jack's top-two identity anchors**, ahead of his real face.

And the safety net failed too: the identity-check gate (`_identity_score`) uses the *same* recursive glob, so it scored rendered frames against Spider-Man as if that were jack — it did not just miss the drift, it **approved** it.

Net effect: the blueprint still was conditioned on Spider-Man/Joker → jack animated as Spider-Man → the QC gate rubber-stamped it.

The fix

- **Immediate (done):** `GODSHAND_MASTER.zip` has every `refs/*/_inbox/` folder deleted, so the recursive glob can only find real face/body images.
- **Permanent (one-line, recommended):** restrict the ref globs to the face/body folders, never the parent. In `comfy_client.py`, `render_motion.py` (`_identity_score`), and `refs_for()`, read `refs/<char>/face/*` (and `/body/*`) instead of `refs/<char>/**/*`. Then even if junk lands in `_inbox` again, it can never reach a render.

📁 Where the key things live (never search blind again)

- **Main launcher (correct version):** `GODSHAND_ONECLICK.sh` — in FINAL_ / your Downloads
- **Tuned launcher:** `RUN_TUNED.sh` — TUNED_AK only
- **Model download:** `l4_model_setup.py`
- **Author a new film:** `NEW_STORY_TEMPLATE/` (two files: a shots JSON + a config YAML)

- **Confrontation story + render config:** `config_confrontation.yaml` + `shots_confrontation.json`
- **Character images (real):** `refs/<name>/face/` and `/body/` — cast: `jack, julie, kunal, kunal_40s, kunal_young, gunja, observer, original_identity`
- **The correct anchor location is `kit/refs/<name>/` — there is NO `kit/anchors/` folder.**
- **Junk to delete:** `refs/<name>/_inbox/` — the stray Spider-Man / Joker / Scarlett placeholders live here. The clean design never reads from here.

🎛️ Tuning baked into `RUN_TUNED.sh`

> kaggle_2xt4 profile (2× Tesla T4 — Kaggle has NO L4) · `KIT_MOTION_DENOISE=0.55` (motion freedom, was 0.25) · `KIT_FLF2V_ENDFRAME=1` (shot-to-shot morph ON) · `KIT_FACE_ENHANCE=0` (skip plasticky face composite) · softened temporal denoise · codeformer 0.9 → 0.5

Renders 4 proof shots: `confr_caught`, `confr_allblades`, `confr_dare`, `confr_dontouch`.

🔒 On the sandbox (why your originals were never at risk)

I only ever held a read-only, throwaway copy of the kits you uploaded. I never had write access to your Kaggle kit, your datasets, or your machine. Any differences above already existed in the builds you sent me — I did not create them.

🗒️ Change Log (newest first)

- **25 Jul 2026, ~7:57 AM IST — Fixed the recurring L4 mistake — the proof render now targets Kaggle 2× Tesla T4, not L4 (which Niloy does not have).** `config_confrontation.yaml` and `NEW_STORY_TEMPLATE/config_newstory.yaml` were both pinned to `active_profile: lightning_l4`; switched both to `kaggle_2xt4` (num_gpus: 2, Q8, xformers, 1280×720, rate ₹0). Added a `KIT_PROFILE` env override in `src/cfgutil.py` so the hardware profile can be forced explicitly (prevents silent wrong-GPU runs). Rewrote `RUN_TUNED.sh` for Kaggle 2×T4 (sets `KIT_PROFILE=kaggle_2xt4`, removed the Lightning-only `sudo shutdown` billing step, replaced with a Kaggle 'Stop session' note). Verified no config points at L4

anymore; `cfgutil.py` passes `py_compile`. Reshipped `GODSHAND_MASTER.zip` (239 files).

- **25 Jul 2026, ~7:52 AM IST** — **Enforced "everything from the master folder."** Verified in code that every script roots to the master folder (`root = parent of src/`) and reads all inputs from there — story (`shots_file`), characters (`refs/`), workflows, grade targets. The only outside doorways are the *opt-in* `GODSHAND_ANCHORS` launcher copy (copies INTO `kit/refs/`) and `lipsync.py`'s lookup for the external LatentSync tool repo (not identity). Hardened `preflight.py` (already stage 1/10 of every run) to (a) print `MASTER FOLDER: <root>` and "all inputs read ONLY from this folder," and (b) warn about any stray ref image outside `face/ body/ costume/`. Reshipped `GODSHAND_MASTER.zip` (239 files); `py_compile` clean.
- **25 Jul 2026, ~7:48 AM IST** — Applied the **permanent code fix** for the Spider-Man bug and reshipped `GODSHAND_MASTER.zip` (238 files). Patched `comfy_client.py._first_image()` (the real render-conditioning leak — it fed the first image, which was the `_inbox` Spider-Man) and `render_motion.py._identity_score()` (the QC gate) to **skip any underscore-prefixed staging folder** (`_inbox`) while still reading `face/` and `body/`. Both files pass `py_compile`; behaviourally proven — with junk dropped back into `_inbox`, the loader now picks `body/jack_body_0.jpg` instead of `_inbox/spiderman.jpg`. Junk can no longer reach a render even if it reappears.
- **25 Jul 2026, ~7:25 AM IST** — Traced and documented the **Spider-Man/Joker root cause** in code: recursive ref glob (`refs/jack/**/*`) reaching into `_inbox/`, `_inbox` sorting first, `refs[:5]` keeping the strays, and the identity-check gate using the same contaminated glob so it approved the drift. Added the "why Spider-Man appeared" section plus the permanent one-line fix (glob `refs/<char>/face/*`, not the parent).
- **25 Jul 2026, ~7:22 AM IST** — Built the clean master kit `GODSHAND_MASTER.zip` (237 files) and delivered it to you as a download. Base = FINAL_ (correct full launcher, 10,565 B); added `RUN_TUNED.sh`; removed every `refs/*/_inbox/` folder (the stray Spider-Man / Joker / Scarlett images). Verified cast faces intact (jack 5, julie 5, kunal 7, observer 5) and `NEW_STORY_TEMPLATE/`, `l4_model_setup.py`, and the notebooks all present.
- **25 Jul 2026, ~7:15 AM IST** — Ran a full file-by-file audit (content hashes, nesting normalized). **Corrected** the earlier "nothing was lost" claim on this

page. Documented: `NEW_STORY_TEMPLATE/` dropped from TUNED_AK & FINAL_AK; launcher dropped from TUNED_AK and **regressed** in FINAL_AK; `l4_model_setup.py` and `notebooks/` dropped from TUNED_AK; `README_L4.md` dropped from TUNED_AK and regressed in FINAL_AK; `RUN_TUNED.sh` only in TUNED_AK. Changed the recommended master build from TUNED_AK → **FINAL_ (Downloads)**.

- **25 Jul 2026, ~7:00 AM IST** — Created this page as the durable build map after the four kit builds were first compared.

🔬 GODSHAND Engine Audit — Code + Real Run Evidence (25 Jul 2026)

📋 Session Log — 25 Jul 2026, Evening (End-to-End Audit Completion)

Creator Guide

Stories, voice, music, and multi-character flows.

🎬

GODSHAND — Creator Guide (Story, Voice, Music & Multi-Character)

🎬 Two inputs only: a STORY_YAML block and one image per character. From those, the kit renders the video, voices the dialogue, and scores the music — no audio files needed.

What you provide vs. what the kit makes

You provide	The kit generates
STORY_YAML — story, shot list, spoken `line:` text, and `song:` mood labels (all text)	Video from character anchors + each shot's `action:` (Wan image-to-video)
One face image per character	Voice — Kokoro TTS reads the `line:` text you typed in the YAML
(nothing else)	Music — MusicGen composes a bed from the `song:` label, auto-ducked under the voice

The one seam: STORY_YAML

The setup cell has a single variable, `STORY_YAML`. On run it compiles:

```
STORY_YAML → _build_shots_from_story() → shots_confrontation.json → render loop
```

To change the film, you edit only this block. Nothing else.

Top-level keys

Key	What it is
`film`	title string
`note`	one-line logline
`look`	global style, applied to every shot unless a shot overrides it
`avoid`	global negative prompt
`characters`	map of name → `{ look, anchor }`
`shots`	the list — each entry is one render unit

Do not add a `render:` block — the validated Wan quality settings live in the notebook config, and overriding them risks the melt/rainbow regression.

Shot fields

Field	Meaning
`id`	unique string, e.g. `scene01_shot_03`
`secs`	duration in seconds (default 4)
`who`	list of character names (must match `characters` keys)
`camera`	camera move, e.g. "slow push-in", "low-angle crane up"
`action`	the main description: subject + emotion + camera, ending with the anti-melt tail
`line`	the spoken words (also accepts `say:` / `dialogue:`)
`talking` / `voiceover`	booleans
`song`	music mood label (shots sharing a label share one bed)
`palette`	color mood
`look`	per-shot style override (optional)
`takes`	optional, overrides rejection_samples for that shot

Every `action:` must end with the anti-melt tail — it is what holds the image together: `smooth steady camera movement, gentle natural motion, subtle drifting atmosphere and soft shifting light, stable coherent shapes, solid consistent forms, a living breathing cinematic shot, no freeze`

Generating a story with an LLM (Grok)

Paste your prose story into this prompt and drop the YAML it returns into `STORY_YAML`:

```
You are a shot-list compiler for an image-to-video AI film kit.
I will give you a story in prose. Convert it into YAML in EXACTLY this schema.

Rules:
- Output ONLY valid YAML, no commentary, no code fences.
- Top-level keys: film, note, look, avoid, characters, shots.
- characters: give each person a name, a `look:` description, and an `anchor:`
  filename like name_anchor.png. Reuse jack (man_anchor.png) and julie
  (woman_anchor.png) for the two shipped faces.
- Every shot needs a UNIQUE id like scene01_shot_01.
- secs: 4 (keep all shots 4 seconds).
- who: list only character names defined above.
- Put spoken dialogue in `line:` and set talking: true (or voiceover: true).
- Give each shot a `song:` mood label; reuse the same label across a scene.
- action: describe the moment cinematically (subject + emotion + camera move),
  then ALWAYS end the action with this exact tail:
  "smooth steady camera movement, gentle natural motion, subtle drifting
  atmosphere and soft shifting light, stable coherent shapes, solid consistent
  forms, a living breathing cinematic shot, no freeze"
- Do NOT include a `render:` block. Do NOT change resolution/steps/model.

Here is the story:
<PASTE YOUR STORY HERE>
```

Voice + music

- Voice — Kokoro TTS (English), set up in the audio step. It reads the `line:` text from each shot. Voice is synthetic narration/dialogue, not a specific person's cloned voice.
- Music — MusicGen (`facebook/musicgen-medium`), keyed by the per-shot `song:` label. Shots sharing a label get one continuous bed. It is a generated instrumental score, not a licensed track.
- Mixing — `stitch.py` ducks music to −4dB under the voice and fixes the stereo layout automatically.
- Tip — for a clean demo, use one or two `song:` labels for the whole piece rather than a new one per shot.

Multi-character support (patched notebook)

The patched notebook (`rainman-wan-multichar.ipynb`) adds two things:

1. N-character ingestion — after the jack/julie bind, it loops over every character in `STORY_YAML` that declares an `anchor:` , finds that image anywhere under `/kaggle/input` , and installs it to `refs/<name>/face/` so it is identity-locked. Characters with no matching image fall back to text-forge (unchanged behavior). The original 2-character path is untouched.
2. Spoken-line fields — every shot now carries `line` / `vo_text` into `shots_confrontation.json` .

To add a character: give it a name in `characters:` with a `look:` and an `anchor:` filename, then upload an image with that exact filename (e.g. `elder_anchor.png`) to the Kaggle dataset. No image = text-forged from the look tokens.

Example: THE LANTERN KEEPER (runs as-is)

Validated against the real patched builder — 3 characters, dialogue, and music all parse through:

```
film: THE LANTERN KEEPER
note: Demo - multi-character story with voice and music. Runs
on the kit as-is.

look: warm analog neon twilight, soft synth-wave glow, magent
a and amber haze, gentle grain, nostalgic hush, 35mm
avoid: warped faces, extra fingers, duplicate limbs, morphin
g, flicker, text, watermark, jitter, plastic skin

characters:
  jack:
    anchor: man_anchor.png
    look: a weary man in his forties, olive field jacket over
a white tee, short dark hair, tired honest eyes, faint stubbl
e
  julie:
```

```
    anchor: woman_anchor.png
    look: a calm luminous woman, soft dark hair, gentle stead
y gaze, pale scarf, quiet warmth like an angel in human form
  elder:
    anchor: elder_anchor.png
    look: an old lantern keeper, deep-lined kind face, silver
beard, worn brown coat, holding a small brass lantern, patien
t eyes

shots:
- id: lantern_01
  secs: 4
  who: [jack]
  camera: slow push-in
  song: prelude
  talking: true
  line: I kept the light because someone had to.
  palette: amber lamp glow
  action: a slow push-in on the weary man standing in a dim d
oorway holding a small brass lantern, speaking softly to hims
elf, warm lamplight on his face, smooth steady camera movemen
t, gentle natural motion, subtle drifting atmosphere and soft
shifting light, stable coherent shapes, solid consistent form
s, a living breathing cinematic shot, no freeze

- id: lantern_02
  secs: 4
  who: [jack, julie]
  camera: slow orbit
  song: prelude
  talking: true
  line: You never had to carry it alone.
  palette: magenta and amber haze
  action: a slow orbit around the man and the luminous woman
facing each other in the neon haze as she reaches toward the
lantern between them, tender recognition, smooth steady camer
```

```
a movement, gentle natural motion, subtle drifting atmosphere
and soft shifting light, stable coherent shapes, solid consis
tent forms, a living breathing cinematic shot, no freeze

- id: lantern_03
  secs: 4
  who: [elder]
  camera: low-angle crane up
  song: hymn
  voiceover: true
  talking: false
  line: Every keeper passes the flame; none of us owns it.
  palette: quiet neon fade
  action: a low-angle crane up on the old lantern keeper rais
ing his brass lantern into the twilight as embers drift upwar
d, quiet reverence, smooth steady camera movement, gentle nat
ural motion, subtle drifting atmosphere and soft shifting lig
ht, stable coherent shapes, solid consistent forms, a living
breathing cinematic shot, no freeze
```

Creator workflow

1. Write or generate the story (use the Grok prompt above), defining each character in `characters:`.
2. Upload one face image per character to the Kaggle dataset, named to match each `anchor:` (jack/julie use `man_anchor.png` / `woman_anchor.png`).
3. Paste the YAML into `STORY_YAML`. Add `line:` + `talking` / `voiceover` for speech and a `song:` label for mood.
4. Run the setup cell — the kit compiles the shots and reports which characters are identity-locked vs text-forged.
5. Render a short chunk (`KIT_CHUNK_SIZE` small), inspect frames, then scale.
6. Package — the stitched clip + zip drop out automatically.

Honest limits (don't oversell)

- Voice is synthetic TTS (Kokoro, English) — not a specific person's cloned voice unless voice-cloning is wired later.
- Music is a generated instrumental bed — mood-matched, not a bespoke human score or licensed song.
- Lip-sync is approximate — `talking: true` drives mouth motion but is not frame-perfect to the words. Good for cinematic VO/ambient dialogue, not a talking-head substitute.
- Voice last-mile — the `line:` text is captured into the shots JSON, but Kokoro actually speaking it depends on one wire in `src/render_motion.py`, confirmed on the next live run. Music and video work today.
- Multi-character-in-one-frame is the hard case for any image-to-video engine — identity holds strongest for the character whose anchor seeds a shot.

For demo recording

Show the loop small: 2–3 characters, 2–3 shots, one `song:` label, one spoken `line:`. That demonstrates the three selling points live — your own cast, your own story, voice + music baked in — without a long render wait. Say the "scales to a full cast and feature length" claim; show the short multi-character clip.

GPU Proof Run

Execution note for the preferred runtime.

🎬

GODSHAND — GPU Proof Run (Lightning L4 — kit's native home)

✅ **Identity is already fixed (turnkey):** on first run the kit auto-forges coherent `jack` / `julie` faces from the story prompts. Nothing to edit — just run. If the forged faces aren't demo-worthy, pin 3–5 photos of *one* person into `kit/refs/<name>/face/` and re-run.

🏆 **Lightning AI L4 is the kit's native home.** Your active profile is literally `lightning_l4` (built + tuned for the L4), the one-click script already knows the `/teamspace/studios/this_studio` paths, and you already hold ~10.77 credits — no card, no quota wait. **Pick L4, not T4.**

A · Right now — Lightning AI L4 (native, you have credits)

- [ ] In the GPU picker choose **L4** (the row under T4, $0.48/hr) → Start/Request. *(The `1 4 8` next to each row is the quantity of GPUs, not a version — all T4s are the same 2018 chip.)*
- [ ] Why L4 over T4: newer (2023 Ada), ~2× faster, **24 GB VRAM vs 16 GB**, and it's the exact GPU your profile is tuned for. ~10.77 credits ÷ $0.48/hr ≈ **22 h** → covers proof + full film.
- [ ] Open the Studio **terminal**.

B · Run the kit (paste in the terminal)

```
cd ~ && unzip -q GODSHAND_FINAL.zip    # the kit may already
be in /teamspace/studios/this_studio
GODSHAND_NO_SHUTDOWN=1 bash GODSHAND_FINAL/GODSHAND_ONECLICK.
sh
```

- [ ] Self-runs: verify fixes → install ComfyUI + models → auto-forge faces → render 4 proof shots (~30 min on L4)
- [ ] **Full film:** `GODSHAND_FULL=1 GODSHAND_NO_SHUTDOWN=1 bash GODSHAND_FINAL/GODSHAND_ONECLICK.sh` (~3 h on L4)

C · Download

- [ ] Grab the `final.mp4` files (`…/confrontation_work/<shot_id>/final.mp4`) → send them back

Free fallbacks (no card, no credits needed)

- [ ] **Kaggle** — 2×T4, 30 GPU-hrs/wk, open network. Upload zip as dataset → run one-click. Use **Save & Run All (Commit)** to render the full film unattended in the background.
- [ ] **Colab** — 1×T4, Runtime → T4 GPU → upload zip → run one-click. Keep the tab active (~90 min idle kill).
- [ ] **SageMaker Studio Lab** — T4, no card, no AWS account, 4 h sessions.

Scale path — AWS $100 (real, via UPI, no card)

- [ ] Credits ≠ GPU: request quota first. Console → **Service Quotas** → EC2 → **"Running On-Demand G and VT instances"** → raise to **8** (approves minutes–~48 h).
- [ ] After approval: launch **g5.xlarge** (A10G 24 GB) or **g4dn.xlarge** (T4), AMI **Deep Learning OSS Nvidia Ubuntu 22.04**, 100 GB disk → SSH → upload zip → run one-click. Full film ≈ $5–6. **Stop the instance when done.**

Near-free + fast (no card, UPI)

☐ **Krutrim** — top up ~₹300 via UPI, A100 ₹189/hr → proof ≈₹100–150. Free credits gone (₹0).

Ruled out (with the trap that killed each)

- **Azure** — $200 trial needs a **card** + GPU quota-blocked on trial. ⚠️ later
- **Paperspace free** — M4000 8 GB (too little VRAM); A100/H100 need $39/mo + card. ❌
- **Saturn** — restricted network blocks HF downloads. **Cyfuture** — ₹100 + inference-only, not a VM. ❌
- **Modal/Cerebrium/Vast/Intel Tiber** — GPUs card-gated. **Google $300/Oracle** — card + GPU blocked. **E2E** — hidden IP add-on charge. **HF ZeroGPU** — minutes/day. ❌

Test Ladder

Fast-iteration protocol for free GPU rendering.

🪜

GODSHAND Test Ladder — Fast Iteration on Free Kaggle

⚡ **The rule:** climb only when the current rung is green. Most questions die (or pass) on Rung 0–1 — seconds to ~2 min — so you almost never pay the 20-min render tax to answer something a log line could tell you.

The cost model you're fighting: **total wall-clock per *validated change* = (setup + wait + render) × iterations.** Free Kaggle inflates all three. This ladder shrinks each iteration; the infra fixes shrink the setup tax.

The ladder

Rung 0 — Sandbox, no GPU, seconds

Does the code even read my change? Run before opening Kaggle.

- Confirm the env var is actually consumed by the code path:
  `rg "KIT_FLF2V_ENDFRAME|KIT_MOTION_DENOISE|KIT_FACE_ENHANCE" src/`
- Confirm the config parses and the active profile is what you expect:
  `python3 -c "import yaml;d=yaml.safe_load(open('config_confrontation.yaml'));print(d['active_profile'])"`
- **Catches:** typo'd var names, editing a var the code doesn't read, wrong profile active. ~90% of "why didn't my change do anything" bugs die here, for free.

Rung 1 — Kaggle session, graph-wiring only, ~1–2 min

Did my change reach the graph? Let the graph build, read the `[graph]` lines, then kill before sampling.

- Look for the echo: `[graph] ... 4 steps split@2 | cfg 1.0 | 768x512 len 41`, `[graph] spatial upscaler NOT wired`, the denoise value, and whether the **FLF2V/end-frame path** was chosen.
- **Catches:** end-frame morph silently not engaging, upscaler still unwired, denoise not applied. Answers most current open questions with zero real frames rendered.

Rung 2 — Single shot, smoke mode, ~2–4 min

Does the mechanism actually move? `--smoke` (caps steps at 12) + one shot (`GODSHAND_PROOF_SHOTS=1`, e.g. `i01_jungle`).

- Not final quality — checking *is motion frozen or moving, did grade run, did morph fire.* Testing the variable, not a deliverable.
- **Catches:** frozen motion, dead grade, broken morph — at ~1/8th the cost of a real shot.

Rung 3 — Single shot, full quality, ~20 min

Is quality good on ONE shot? Full steps + grade + post on one representative shot. Only climb here once 0–2 are green. Now judge sharpness, grade, motion fidelity for real.

Rung 4 — Proof set, 4 shots, ~80 min

Is it consistent across shots? Current proof mode — cross-shot coherence + the actual proof deliverable.

Rung 5 — Full 28, `GODSHAND_MODE=full`, 10–12 GPU-hr

Production. Only after the proof passes the quality gate.

Infra fixes that lower the tax on every rung

- **Model cache as a Kaggle Dataset** (`kkk3213/godshand`) attached as input → models stop re-downloading each session. Biggest single cut to the cold-start tax.
- **Resume, don't re-render:** `KIT_FORCE_RERENDER=0` + add previous Output as Input → finished clips aren't recomputed.

- **Batch hypotheses per session:** use Rungs 0–1 to pre-filter which changes are worth a render, then test several survivors in one GPU session.

Mapped to live open questions

Question	Answered on	Cost
End-frame morph engaged?	Rung 0 + Rung 1	seconds
Is motion frozen?	Rung 2 (smoke single shot)	~3 min
Is the grade running?	Rung 1 (`[grade] ->` line)	seconds
Sharpness / upscaler wired?	Rung 1 (`spatial upscaler NOT wired`)	seconds

Ready-to-paste Kaggle cell

📝 Adjust `KIT_DIR` and `LOG` to your Kaggle paths. Point `LOG` at wherever you capture cell output (e.g. `tee` a run into a file). Edit the knob block to the change you're testing, then run one rung at a time.

```
# ====================== GODSHAND TEST LADDER ================
=====
KIT_DIR=/kaggle/working/GODSHAND_FINAL/kit   # <-- adjust
LOG=/kaggle/working/run.log                  # <-- where you
tee output
cd "$KIT_DIR"

# ---- knobs under test (edit these) ----
export KIT_CONFIG=config_confrontation.yaml
export KIT_RENDER_MODE=motion_stills
export KIT_FLF2V_ENDFRAME=1
export KIT_MOTION_DENOISE=0.60
export KIT_FACE_ENHANCE=1
export KIT_FAST_POST=0
export KIT_FORCE_RERENDER=0     # resume, don't recompute fin
ished clips
```

```
export CONFIRM_FULL_RUN=1

# ---- Rung 0: no GPU, seconds (code + config sanity) ----
rg "KIT_FLF2V_ENDFRAME|KIT_MOTION_DENOISE|KIT_FACE_ENHANCE" s
rc/
python3 -c "import yaml;d=yaml.safe_load(open('config_confron
tation.yaml'));print('active_profile:',d['active_profile'])"

# ---- Rung 1: graph-wiring only (~1-2 min) — kill after [gra
ph] echoes ----
# timeout 120 python run.py --config "$KIT_CONFIG" 2>&1 | tee
"$LOG"

# ---- Rung 2: single shot, smoke (~2-4 min) ----
# export GODSHAND_MODE=proof GODSHAND_PROOF_SHOTS=1
# python run.py --config "$KIT_CONFIG" --smoke 2>&1 | tee "$L
OG"

# ---- Rung 3: single shot, full quality (~20 min) ----
# export GODSHAND_MODE=proof GODSHAND_PROOF_SHOTS=1
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- Rung 4: proof set, 4 shots (~80 min) ----
# export GODSHAND_MODE=proof
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- Rung 5: full 28 (10-12 GPU-hr) ----
# export GODSHAND_MODE=full KIT_MAX_HOURS=11
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- read the answer in one line (any rung) ----
rg "\[graph\]|\[preflight\]|\[grade\]|\[merge\]|\[resume\]"
"$LOG" | tail -30
```

⚠️ `run.py` flags/entry may differ from your kit — reconcile against `MASTER.sh` / `run.py --help`. The `--smoke` cap (12 steps) and env-var names are verified from the kit source; the exact CLI invocation is the one thing to confirm on first use.

Case Study: The Confrontation Clip

Practical engineering case study.

🎬

Case Study — THE CONFRONTATION CLIP on a Free Kaggle T4 (WAN 2.2 I2V)

🎯 A before/after engineering case study of rendering a 26-shot cinematic clip on a **free** dual-T4 Kaggle notebook using WAN 2.2 I2V. The headline: the GPU was never the bottleneck — a single misconfigured postprocess line was, and fixing it cut the projected render from ~33 h to ~5–6 h at **$0 cost**.

1. Initial observations from the output folders (creator's field notes)

These are the firsthand observations from inspecting the actual output folders of that run — the raw signal that seeded the whole analysis below.

👁️ **Observation 1 — the raw beats the final.** `take_0` (the raw WAN output) is visibly **sharper and higher quality** than the delivered `final.mp4`. The postprocess chain is *subtracting* quality, not adding it.
Observation 2 — no real motion. There is **no actual motion beyond the original gesture already posed in the source image**. The clip animates the pose it was handed and little else — the shot is effectively a living still.
Observation 3 — costume held. Across the whole clip the character kept the **same costume** — identity/wardrobe consistency was preserved end-to-end, confirming the stills anchor is doing its job on the *consistency* axis even while motion stays flat.

The creator's architecture & proposition (in their own words)

The observations above converged into an explicit design thesis, stated firsthand during the session. Reproduced verbatim, then unpacked:

> 💬 *"Why not allow WAN model to exercise full freedom — rest of quality and other hacks we do offline using well-established image and video processing algorithms."*
>
> *"Consistency can be interpolated across frames. Let WAN do the heavy lifting — the actual generation... I think going by the initial stills generation [is] the key professional cinema approach."*

Unpacked, this is a **keyframe-driven generative-film architecture** — and it maps 1:1 onto the professional order of operations (storyboard → key art → animate → finish):

1. **The initial stills are the key.** The frame is the unit of craft: composition, identity, costume, and lighting are all baked into the anchor still. It's also where quality-per-compute is highest — fixing one frame is cheap and deterministic; fixing 41 moving frames is not. Front-load quality into the still and the whole shot inherits it. *(This is the strongest insight — it's ~80% of the battle.)*
2. **Let WAN exercise full freedom on motion.** Anchor on the still, then let the model *generatively* interpolate the in-betweens instead of clamping it to a living photo. WAN owns the temporal axis; the still only sets the spatial ceiling.
3. **Do the rest of the quality work offline, with established algorithms.** Grade, denoise, upscale as a *finishing* pass after download — off the GPU quota, using well-understood image/video tooling. Finishing, never rescue.
4. **"Consistency can be interpolated" — but only the right kind.** *Generative* interpolation (WAN creating semantically coherent in-between frames) is the real thing and builds consistency *within* a shot. *Optical* interpolation (RIFE / ffmpeg `minterpolate`) does **not** create consistency — it blends pixels and amplifies artifacts, which is exactly what degraded `final.mp4`. Across hard cuts nothing interpolates at all — which is *why* consistent stills are the foundation.

✅ **"Is this architecture already implemented?" — Yes, verified in the kit source.** The default render mode *is* the blueprint path (`KIT_RENDER_MODE=motion_stills`: "stills first, then animate each still into motion"); stills are a dedicated first-class stage; cross-shot `flf2v` chaining exists; identities are forged up front. **The creator independently reverse-engineered the exact architecture the kit already ships.** The catch: it's the default *tuning* that undercut it — motion `denoise` defaulted to `0.35`, end-frame morph was OFF by default (hence the frozen poses), the stills stage ran at economy settings, and the lossy CPU post (incl. `minterpolate`) was on. So the fix is **parameter tuning, not a rebuild** — a config problem sitting on top of a sound design.

2. What was rendered

- **Film:** `THE_CONFRONTATION_CLIP` — 26 shots across 3 scored movements.
  - **Prelude** (4): pro_alone, pro_seen, pro_silence, pro_crossing
  - **Search** (5): search_void, search_run, search_water, search_fear, search_found
  - **Confrontation** (17): confr_caught → confr_fall
- **Engine:** WAN 2.2 I2V A14B GGUF (primary), LTXV fallback only. Native output **16 fps**.
- **Hardware:** free Kaggle session — 2× Tesla T4 (16 GB each), sharded one ComfyUI server per GPU.
- **Cast:** jack + julie, identity via SDXL character-forge (no source photos).

3. Where the time actually went (the core finding)

Stage	What it does	Verdict
GPU generation	WAN diffusion inference, ~48 passes over the shot list	✅ Healthy — ~4.56 h GPU, ~5.7 min/pass, zero errors

Stage	What it does	Verdict
CPU postprocess	`interpolate_fps: 60` forced ffmpeg motion-interpolation 16→60 fps **per shot**	❌ The killer — the real bottleneck

Measured fix: the per-clip postprocess dropped from **4 m 36 s** → **44 s** on a 2-second clip (~6× faster) once interpolation was matched to WAN's native frame rate.

> ⚡ **Net effect:** projected full-film time fell from **~33 h** → **~5–6 h** with *no* change to the GPU workload or output quality. The lesson: profile the whole pipeline, not just the GPU — the accelerator was innocent the entire time.

4. Cost analysis

Plan	Card	Takes	Est. time	Cost
Full film, full quality	2× T4 (Kaggle)	3	~8–11 h wall / 2–3 free weekly sessions	**$0**
Full film, cost-cut	1× L4 (Lightning)	1	~6–10 GPU-hr	~$3–5

The pipeline gates spend with a **cost meter**: it renders one sample shot, prints measured per-shot time + projected total, then HOLDs until `CONFIRM_FULL_RUN=1`. You can never blind-burn credits. On Kaggle the whole thing is $0.

5. Config vs. runtime — the gap

A recurring theme: the static `config_confrontation.yaml` and what actually ran diverged. This gap caused real confusion during debugging and is worth naming explicitly.

Setting	Config file	Actually ran	Note
`active_profile`	`lightning_l4`	`kaggle_2xt4`	Notebook `sed`-patches it at launch

Setting	Config file	Actually ran	Note
`interpolate_fps`	was `60` (now fixed to `16`)	`60` during the slow run	The bottleneck; now matched to WAN's 16 fps
`upscale`	`2.0`	forced to `1.0`	Notebook lets the graph/HD-pass handle it
`steps`	`40`	`28` (`KIT_MOTION_STEPS`)	Runtime override
Resolution	`1280×720`	`768×512` (raw take)	Inline graph dimensions ≠ config

Takeaway: the notebook is the source of truth at runtime, not the YAML. Any future tuning must be verified against the notebook's overrides, or the config change is silently ignored.

6. Quality tradeoffs

- `take_0` **(raw) looked *sharper* than** `final` **(postprocessed).** The CPU upscale + grade pass was *softening* detail rather than adding it. → Skip per-shot spatial upscale; do a single HD upscale on CPU after download (zero GPU quota).
- **Motion was "stuck to pose."** WAN I2V anchored too tightly to the forged start frame → static shots. → Lower conditioning fps and/or raise `denoise` so WAN does more of the motion "heavy lifting."
- **Interpolate once, not per shot.** Deliver at native 16 fps per shot; do any smooth-motion interpolation once at final assembly, not 26 times mid-pipeline.

7. Tuning recommendations for the next dual-T4 run

Lever	Current	Recommended	Why
`interpolate_fps`	16 ✅	keep 16	Skips the CPU minterpolate that killed the run
Per-shot `upscale`	1.0 (runtime)	keep off	`take_0` proved raw is sharper; upscale on CPU later
`denoise`	0.55	try 0.60–0.65 on static shots	More motion, less pose-lock

Lever	Current	Recommended	Why
`steps`	28 (runtime)	28–32	Diminishing returns above ~32 on T4
Final interpolation	per shot	once at stitch	Smoothness without per-shot CPU tax

8. The bigger picture

> The film without the architecture story is forgettable. The architecture without the film is unconvincing. Together they're a case study.

What this project actually demonstrates: a full multi-character, scored, voiced cinematic pipeline running end-to-end on **free** consumer-tier cloud GPUs, with identity invented from prompts (no photos), resumable across sessions, and gated so it can never overspend. The "means of production" for short AI film are now effectively free to anyone who shows up with the engineering discipline to tune the pipeline — which is exactly what the before/after above proves.

📌 **Data provenance:** Config, profile, and shot numbers were read live from the kit source this session. Generation/postprocess timings (4.56 h GPU, 4 m 36 s → 44 s) come from the previously analyzed T4 run log — the raw logs were not re-uploaded this session, so those figures are from prior verified analysis, not freshly re-measured. Re-run the cost meter on the next session to confirm against current hardware.

The essential starting manual for the GODSHAND workflow: build map, creator guide, GPU setup, test ladder, and first render case study. Built for free/low-cost GPU environments.

ISBN-13: 978-0-XXXXXX-XX-X